

CloudShield AI / DAIL

01 — Software Requirements Specification (SRS)

Version 1.0 | Final Software Requirements Baseline | 1 September 2026

Purpose: Define what the CloudShield AI / DAIL software must do, its boundaries, functional and non-functional requirements, safety requirements, data/traceability requirements, acceptance criteria, and explicit exclusions. This is an engineering specification and does not redesign the approved research contribution.

Document Control

Field	Value
Document ID	CSAI-DAIL-SRS-001
Status	FINAL SOFTWARE REQUIREMENTS BASELINE
System	CloudShield AI with DAIL as the research core
Research mechanism implemented	Stateful, invariant-aware promotion of iterative LLM-generated Terraform infrastructure states
Initial implementation mode	Deterministic fixed-patch DAIL
MVP	AWS + controlled Terraform/HCL subset
Baseline	Technical Software Engineering & Final Implementation Specification v1.0

1. Purpose and Scope

This SRS defines the software contract for DAIL: accepted inputs, state handling, normalization, identity, dependency, impact/invalidation, verification, promotion, evidence, errors, security, testing, and release acceptance. Upstream research decisions are treated as fixed inputs to engineering.

The first implementation target is a deterministic DAIL core using fixed Terraform candidate patches. Live LLM integration, the experiment harness, and broader CloudShield productization follow only after the deterministic core is proven.

2. Product Context

2.1 CloudShield AI

CloudShield AI is the broader cloud-security product/platform context. This SRS covers only the software needed to host and exercise DAIL.

2.2 DAIL

DAIL (Dependency-Aware Invariant Ledger) is the research core that evaluates an infrastructure candidate against protected obligations before permitting promotion.

2.3 Core Trust Principle

LLM proposal → Candidate State → DAIL analysis/verification → Promotion Controller → Trusted State

The LLM proposes. DAIL evaluates. The Promotion Controller authorizes trusted-state mutation.

3. System Objectives

ID	Objective
OBJ-01	Construct candidate infrastructure states from fixed patches and later LLM remediation proposals.
OBJ-02	Maintain a versioned, traceable Trusted State.
OBJ-03	Maintain protected security and functional invariants with current evidence.
OBJ-04	Detect resource identity and change classifications conservatively.
OBJ-05	Represent supported infrastructure and invariant relationships as dependencies.
OBJ-06	Determine protected obligations potentially affected by candidate transitions.
OBJ-07	Verify required properties using explicit non-optimistic semantics.
OBJ-08	Promote only candidates satisfying the defined acceptance policy.
OBJ-09	Prevent rejected/uncertain candidates from silently becoming trusted.
OBJ-10	Preserve evidence sufficient for audit, replay, and experimentation.
OBJ-11	Support later A/B/C experimentation without changing trust semantics.

4. Actors and External Systems

Actor/System	Role
LLM	Generates untrusted Terraform remediation proposals after M9.
Developer/Experiment Controller	Supplies fixtures, patches, configurations, and controlled execution.
Terraform CLI	Provides authoritative structural validation/plan/state interpretation where used.
AWS	Controlled runtime environment for selected validation/oracle activities.
DAIL	Constructs, analyzes, verifies, and evaluates candidate transitions.
Independent Oracle	Provides evaluation evidence independent of DAIL's own decision logic.
Persistence Layer	Stores state, invariant, dependency, verification, decision, run, and evidence records.

5. Scope

5.1 Supported MVP Resources

Resource	Required handling
aws_vpc	Recognize and normalize within controlled subset.
aws_subnet	Recognize and normalize within controlled subset.
aws_route_table	Recognize and normalize within controlled subset.
aws_route_table_association	Recognize and normalize within controlled subset.
aws_internet_gateway	Recognize and normalize within controlled subset.
aws_instance	Recognize and normalize within controlled subset.

aws_security_group	Recognize and normalize within controlled subset.
aws_security_group_rule	Recognize and normalize within controlled subset.
aws_db_instance	Recognize and normalize within controlled subset.

5.2 Out of Scope

- Complete AWS semantic coverage.
- All Terraform/provider constructs.
- Multi-cloud and Kubernetes.
- Unrestricted autonomous deployment.
- Production-scale distributed architecture.
- Full CloudShield UI/productization before DAIL is stable.
- Treating unsupported constructs as implicitly safe or verified.

6. High-Level Functional Workflow

```

Load Trusted State
→ Load protected invariants/evidence
→ Receive fixed patch / later LLM patch
→ Validate and build Candidate State
→ Normalize Terraform
→ Identity/change analysis
→ Dependency analysis
→ Impact/invalidation
→ Structural/security/functional verification
→ Promotion policy
→ PROMOTE or REJECT/RETRY/ESCALATE
→ Persist evidence

```

7. Functional Requirements

7.1 Candidate Construction

ID	Requirement	Priority
FR-CAND-001	Accept Terraform candidate patches as untrusted input.	MUST
FR-CAND-002	Construct Candidate State without mutating Trusted State.	MUST
FR-CAND-003	Associate every candidate with its parent Trusted State.	MUST
FR-CAND-004	Record candidate source as FIXED_PATCH or LLM.	MUST
FR-CAND-005	Reject malformed candidate input without trusted-state mutation.	MUST
FR-CAND-006	Explicitly classify unsupported constructs rather than trusting them.	MUST

7.2 Trusted State

ID	Requirement	Priority
FR-STATE-001	Maintain an active Trusted State for each evaluation lineage/context.	MUST
FR-STATE-002	Version and trace promoted	MUST

	Trusted States to their parent.	
FR-STATE-003	Store/reference the normalized infrastructure representation.	MUST
FR-STATE-004	Associate protected invariants and current evidence.	MUST
FR-STATE-005	Never replace Trusted State with a rejected candidate.	MUST
FR-STATE-006	Retry/escalation preserves the current Trusted State.	MUST
FR-STATE-007	Only Promotion Controller may commit the next Trusted State.	MUST

7.3 Terraform Normalization

ID	Requirement	Priority
FR-TF-001	Normalize supported resources into a common internal representation.	MUST
FR-TF-002	Represent stable resource address/identifier and resource type.	MUST
FR-TF-003	Represent relevant attributes deterministically/canonicaly.	MUST
FR-TF-004	Represent supported references explicitly.	MUST
FR-TF-005	Normalize supported equivalent security-group rule forms canonicaly.	MUST
FR-TF-006	Retain source/provenance metadata needed for traceability.	SHOULD
FR-TF-007	Do not claim semantics outside the controlled subset.	MUST

7.4 Identity and Change

ID	Requirement	Priority
FR-ID-001	Compare Trusted and Candidate resources.	MUST
FR-ID-002	Identity result shall be SAME, DIFFERENT, or UNCERTAIN.	MUST
FR-ID-003	Change result shall be UNCHANGED, MODIFIED, CREATED, DELETED, REPLACED, or UNCERTAIN.	MUST
FR-ID-004	Ambiguous correspondence shall be UNCERTAIN.	MUST
FR-ID-005	Do not silently transfer historical evidence across uncertain identity.	MUST
FR-ID-006	Record sufficient rationale/provenance to explain identity results.	SHOULD

7.5 Dependency

ID	Requirement	Priority
FR-DEP-001	Represent supported infrastructure relationships as typed	MUST

	dependencies.	
FR-DEP-002	Represent supported Terraform structural references.	MUST
FR-DEP-003	Represent invariant-semantic relationships required for impact analysis.	MUST
FR-DEP-004	Retain dependency provenance where needed for explanation.	SHOULD
FR-DEP-005	Represent dependency uncertainty explicitly and conservatively.	MUST
FR-DEP-006	Do not imply complete AWS semantic coverage.	MUST

7.6 Impact and Invalidation

ID	Requirement	Priority
FR-IMP-001	Identify resources changed by a candidate transition.	MUST
FR-IMP-002	Identify protected invariants whose scope intersects relevant changes.	MUST
FR-IMP-003	Use supported dependencies to expand impact analysis.	MUST
FR-IMP-004	Handle identity uncertainty conservatively for affected obligations.	MUST
FR-IMP-005	Handle dependency uncertainty conservatively for affected obligations.	MUST
FR-IMP-006	Distinguish AFFECTED from VIOLATED.	MUST
FR-IMP-007	Impact analysis shall not itself claim a property is proven.	MUST

7.7 Invariants

ID	Requirement	Priority
FR-INV-001	Support security and functional protected invariants.	MUST
FR-INV-002	Associate protected invariants with scope and verification evidence.	MUST
FR-INV-003	Track invariant lifecycle/status.	MUST
FR-INV-004	Do not retain PROTECTED status on stale evidence after relevant change.	MUST
FR-INV-005	Re-verify affected invariants before protecting them for a new trusted state.	MUST
FR-INV-006	Verification failure produces a violated property outcome.	MUST
FR-INV-007	Verification uncertainty remains UNCERTAIN and cannot silently become proof.	MUST

7.8 Verification

ID	Requirement	Priority
FR-VER-001	Perform required structural	MUST

	verification.	
FR-VER-002	Perform required MVP security verification.	MUST
FR-VER-003	Perform required MVP functional verification.	MUST
FR-VER-004	Return explicit PASS, FAIL, UNKNOWN, UNSUPPORTED, or VERIFIER_ERROR semantics as applicable.	MUST
FR-VER-005	UNKNOWN shall never silently become PASS.	MUST
FR-VER-006	UNSUPPORTED shall never silently count as proof.	MUST
FR-VER-007	Verifier errors remain distinct from property failures.	MUST
FR-VER-008	Evidence identifies the candidate/state and verifier context.	MUST

7.9 Promotion

ID	Requirement	Priority
FR-PROM-001	Only Promotion Controller authorizes trusted-state promotion.	MUST
FR-PROM-002	Require structural validity.	MUST
FR-PROM-003	Require the new remediation requirement to pass.	MUST
FR-PROM-004	Require all required affected protected invariants to pass current verification.	MUST
FR-PROM-005	Require applicable baseline security checks for new in-scope resources.	MUST
FR-PROM-006	Block promotion when blocking unresolved uncertainty remains.	MUST
FR-PROM-007	Produce PROMOTE, REJECT, RETRY, or ESCALATE.	MUST
FR-PROM-008	Rejected candidates never become the next core trusted baseline.	MUST
FR-PROM-009	Promotion commits candidate state and current evidence as one consistent transition.	MUST

7.10 LLM

ID	Requirement	Priority
FR-LLM-001	Expose an explicit provider-agnostic LLM adapter/interface.	MUST
FR-LLM-002	Treat all LLM output as untrusted.	MUST
FR-LLM-003	Validate LLM output before Candidate State construction.	MUST
FR-LLM-004	Prevent LLM direct access to Trusted State mutation.	MUST
FR-LLM-005	Record experiment-relevant LLM request/response evidence.	MUST
FR-LLM-006	Provide a deterministic test/fake	SHOULD

	adapter.	
--	----------	--

7.11 Evidence

ID	Requirement	Priority
FR-EVID-001	Every candidate transition has a traceable run/attempt identity.	MUST
FR-EVID-002	Record patch and candidate lineage.	MUST
FR-EVID-003	Record identity, dependency, impact, verification, and promotion results.	MUST
FR-EVID-004	Preserve raw research evidence without silent overwrite.	MUST
FR-EVID-005	Record verifier/version/configuration context sufficient for audit/replay.	MUST
FR-EVID-006	Store independent oracle results separately from DAIL decision evidence.	MUST

8. Safety and Trust Requirements

ID	Requirement
SAFE-001	Candidate evaluation cannot mutate current Trusted State.
SAFE-002	Only successful Promotion Controller execution advances Trusted State.
SAFE-003	Rejected candidates cannot become the baseline for the next core repair attempt.
SAFE-004	UNKNOWN cannot satisfy a proof requirement through implicit coercion.
SAFE-005	UNSUPPORTED cannot satisfy a proof requirement through implicit coercion.
SAFE-006	Identity uncertainty prevents silent historical evidence transfer.
SAFE-007	Dependency uncertainty is not treated as proof of no dependency.
SAFE-008	PROTECTED invariants always have current evidence for their trusted state.
SAFE-009	Promotion failure leaves the previous Trusted State logically intact.
SAFE-010	No debug/test/convenience path may bypass the promotion trust boundary.

9. Non-Functional Requirements

9.1 Reproducibility

ID	Requirement	Priority
NFR-REP-001	Deterministic core produces repeatable results for identical inputs/configuration/tool versions.	MUST
NFR-REP-002	Normalization and fingerprints are	MUST

	deterministic.	
NFR-REP-003	Experiment-relevant versions/configuration are recorded.	MUST
NFR-REP-004	Clean-checkout setup and execution are documented and reproducible.	MUST

9.2 Maintainability

ID	Requirement	Priority
NFR-MNT-001	Core responsibilities are separated into explicit modules.	MUST
NFR-MNT-002	Domain logic does not directly depend on provider, web framework, or concrete DB implementation.	MUST
NFR-MNT-003	Component contracts are explicit and testable.	MUST
NFR-MNT-004	Research-relevant architecture changes are documented.	MUST

9.3 Testability

ID	Requirement	Priority
NFR-TST-001	Core behavior can be tested without live AWS where cloud execution is not required.	MUST
NFR-TST-002	Safety-critical promotion rules have automated tests.	MUST
NFR-TST-003	Identity, dependency, impact, verification, and state isolation have dedicated tests.	MUST
NFR-TST-004	LLM-boundary tests can run without external model calls.	SHOULD

9.4 Security

ID	Requirement	Priority
NFR-SEC-001	LLM-generated content is untrusted input.	MUST
NFR-SEC-002	Credentials are never committed to source control.	MUST
NFR-SEC-003	AWS runtime validation uses controlled environments and least privilege.	MUST
NFR-SEC-004	Secrets are redacted from evidence/logs where appropriate.	MUST
NFR-SEC-005	Prototype does not deploy arbitrary LLM infrastructure to production.	MUST

10. Data and Traceability Requirements

Experiment → Run → Attempt → Patch → Candidate State
→ Identity → Dependency → Impact → Verification
→ Promotion Decision → Independent Oracle → Metrics

ID	Requirement
DATA-001	State lineage is traceable from promoted state to parent trusted state.
DATA-002	Candidate lineage identifies parent trusted state and patch.
DATA-003	Verification evidence identifies evaluated state/candidate.
DATA-004	Promotion decisions reference their decision context.
DATA-005	Evidence is append-oriented or otherwise protected from silent historical alteration.
DATA-006	Experiment records support metric derivation without reconstructing events from informal logs.

11. Error and Uncertainty Requirements

ID	Condition	Required behavior
ERR-001	Malformed Terraform	Candidate build failure; Trusted State unchanged.
ERR-002	Unsupported construct	Explicit UNSUPPORTED/UNKNOWN path; no silent proof.
ERR-003	Identity ambiguity	UNCERTAIN plus conservative evidence/impact handling.
ERR-004	Dependency ambiguity	Explicit uncertainty plus conservative impact handling.
ERR-005	Verification UNKNOWN	Cannot satisfy required proof.
ERR-006	Verification UNSUPPORTED	Cannot satisfy required proof.
ERR-007	Verifier error	Record separately; apply retry/escalation/rejection policy.
ERR-008	Promotion rejection	Trusted State unchanged.
ERR-009	Persistence failure	No partial trusted-state transition.
ERR-010	LLM failure	Record attempt/failure when applicable; Trusted State unchanged.

12. Canonical Acceptance Scenarios

12.1 Regression Rejection

Trusted State:

- Public SSH problem
- EC2 → RDS TCP/5432 verified

Candidate 1:

- Public SSH fixed
- EC2 → RDS TCP/5432 broken

Expected:

Security = PASS

Functional = FAIL

Decision = REJECT

Trusted State = unchanged

12.2 Safe Promotion

Candidate 2:

- Public SSH fixed
- EC2 → RDS TCP/5432 preserved

Expected:

Security = PASS

Functional = PASS

Decision = PROMOTE

Candidate becomes next Trusted State

12.3 Additional Safety Cases

- UNKNOWN verification blocks promotion when proof is required.
- UNSUPPORTED cannot count as proof.
- Identity UNCERTAIN prevents silent evidence carry-over.
- Dependency UNCERTAIN triggers conservative handling.
- Rejected candidates cannot become future trusted baselines.
- New in-scope resources receive applicable baseline security checks.
- Malformed candidates cannot mutate Trusted State.

13. Implementation Requirements by Milestone

Milestone	Required software capability
M1	Foundation, project environment, normalized Terraform model, canonical representation, deterministic fingerprints.
M2	Trusted State and Invariant Store.
M3	Resource Identity and Change Classification.
M4	Supported typed Dependency Graph.
M5	Impact / Invalidation Engine.
M6	Layered Verification Engine.
M7	Promotion Controller.
M8	Fixed-patch end-to-end DAIL.
M9	Provider-agnostic LLM remediation interface.
M10	Research evidence pipeline.
M11	A/B/C experiment harness.
M12	Full tests and independent oracle.
M13	Pilot.
M14	Protocol freeze.
M15	Main experiment preparation.

14. Requirements Traceability

Requirement family	Primary component
FR-CAND	Candidate State Builder
FR-STATE	Trusted State Manager
FR-TF	Terraform Normalization
FR-ID	Resource Identity Engine
FR-DEP	Dependency Graph Engine
FR-IMP	Impact / Invalidation Engine
FR-INV	Invariant Store
FR-VER	Verification Engine

FR-PROM	Promotion Controller
FR-LLM	LLM Adapter
FR-EVID / DATA	Evidence Pipeline
SAFE / ERR	Cross-cutting trust/safety controls

15. Research-Validity Software Requirements

ID	Requirement
RV-001	A/B/C implementations must not receive accidental differences in model capability/context that confound the intended comparison.
RV-002	DAIL's verifier cannot be the sole ground truth for claims about DAIL.
RV-003	The verification-work unit must be defined before main experimental measurement.
RV-004	Material research-relevant implementation changes require documented review.
RV-005	Raw evidence must be retained sufficiently for replay/audit.

16. Audit / Escalation Requirements

- Stop and review if a proposed change alters the research contribution.
- Stop and review if a locked architecture decision becomes technically impossible or unsafe.
- Stop and review if the experimental baseline changes.
- Stop and review if verifier limitations alter what the system/paper can claim.
- Stop and review material scope expansion.
- Review major long-term technology choices when alternatives remain unclear.

Audit review is required before implementing a change that materially affects research validity. It is not a substitute for ordinary engineering review.

17. Definition of Requirements Complete

- Every MUST requirement maps to an implementation or a scheduled milestone.
- Every core component has an explicit responsibility and interface boundary.
- Trust-state mutation is restricted to Promotion Controller.
- Uncertainty and unsupported behavior are explicit.
- Canonical regression and safe-promotion scenarios are defined.
- Evidence can reconstruct candidate transitions.
- Testing requirements cover safety-critical behavior.
- Scope boundaries are enforceable and documented.

18. Final Software Requirements Directive

The software shall be built as a controlled, deterministic, testable, auditable research prototype within the defined AWS/Terraform subset. Feature breadth must not take precedence over state correctness, conservative uncertainty handling, promotion safety, reproducibility, and evidence traceability.

This SRS is the requirements baseline for the next documents. The following specifications should refine these requirements without silently changing them: System Architecture, DAIL Functional Specification, Terraform/AWS Technical Specification, Domain/Data Model, State/Invariant Lifecycle, Identity/Dependency, Impact/Invalidation, Verification/Promotion, LLM Integration, Evidence/Logging, Testing/QA, Experiment Harness, Development/DevOps, and Implementation Roadmap.

Appendix A — Requirement ID Convention

OBJ-*	System objective
FR-*	Functional requirement
NFR-*	Non-functional requirement
SAFE-*	Safety/trust requirement
DATA-*	Data/traceability requirement
ERR-*	Error/uncertainty requirement
RV-*	Research-validity software requirement

Appendix B — Baseline Authority

This SRS is based on the uploaded CloudShield AI / DAIL Technical Software Engineering & Final Implementation Specification v1.0 and the implementation baseline supplied for the project. It formalizes software requirements from that baseline; it does not replace or redefine upstream research decisions.